

**Sistema de detección automática de patrones conversacionales para optimización de
agentes de IA**

Documento técnico con metodología, resultados y conclusiones

Daniel Esteban Alarcón Rojas

Facultad de Ingeniería de Sistemas, Universidad Santo Tomás

Inteligencia Artificial Generativa

Docente Diego Alejandro Vela Beltran

25 de mayo de 2026

Contenido

Resumen	4
Introducción	5
Planteamiento del reto y objetivos	6
Problema identificado	6
Objetivo general	6
Objetivos específicos	6
Metodología de desarrollo	7
Diseño arquitectónico del sistema propuesto	8
Módulos del backend	9
Endpoints principales	9
Modelo de datos e ingesta de conversaciones históricas	11
Pipeline de inteligencia artificial para generación de FAQs	12
1. Limpieza y normalización	12
2. Pares usuario-asistente	12
3. Embeddings semánticos	12
4. Clustering	13
5. Generación con Qwen	13
6. Quality gates	13
Validación humana y frontend de revisión	15

	3
Implementación de ejecución periódica automatizada	17
Evaluación experimental del desempeño	18
Variables de configuración experimental	18
Silhouette pipeline	19
Indicadores observados en pruebas locales	20
Precisión, reducción de ambigüedad y mejora en tasa de resolución	21
Calidad, seguridad y mantenibilidad	22
Resultados obtenidos	23
Conclusiones	24
Recomendaciones y trabajo futuro	25

Resumen

El presente documento técnico describe el diseño, implementación y evaluación de un prototipo de inteligencia artificial para Everwod Technologies, orientado a detectar patrones conversacionales recurrentes en chats de WhatsApp y convertirlos en sugerencias de preguntas frecuentes (FAQs). La solución integra una arquitectura modular en Python/FastAPI, una base de datos PostgreSQL organizada por schemas, técnicas de procesamiento de lenguaje natural, embeddings semánticos multilingües, clustering, generación con modelo local Qwen y un flujo de validación humana. El sistema fue diseñado para funcionar como microservicio escalable, con API unificada, panel frontend de revisión, job programado mensual y mecanismos de seguridad para evitar exposición de información sensible. La evaluación experimental considera métricas de calidad como recurrencia, cohesión, soporte, confianza de generación, alineación pregunta-respuesta, silhouette del clustering y control de duplicados frente a FAQs aprobadas o rechazadas. Como resultado, se obtuvo un prototipo funcional capaz de procesar conversaciones históricas, generar candidatos de FAQs, permitir edición/aprobación/rechazo y persistir decisiones para mejorar futuras corridas. El repositorio para acceder a este prototipo se encuentra en https://github.com/alarconDaniel/everwod_ia_faqs.

Palabras clave: preguntas frecuentes, embeddings, clustering, NLP, FastAPI, PostgreSQL, Qwen, validación humana.

Introducción

Everwod Technologies opera agentes de inteligencia artificial que atienden conversaciones de WhatsApp para múltiples negocios. En este contexto, cada empresa genera consultas repetitivas sobre precios, horarios, domicilios, disponibilidad, pagos, productos, agendamiento y políticas del servicio. Cuando estas preguntas recurrentes no se convierten oportunamente en conocimiento estructurado, el agente puede responder con menor precisión contextual y el equipo humano debe revisar manualmente grandes volúmenes de conversación.

El reto universitario solicita diseñar e implementar un sistema que analice conversaciones históricas, detecte patrones semánticos recurrentes mediante técnicas de NLP y sugiera automáticamente nuevas FAQs, evitando duplicación al considerar las existentes, integrando modelos locales bajo una arquitectura escalable y evaluable experimentalmente. La solución desarrollada responde a ese alcance mediante un microservicio de sugerencias que procesa datos históricos, agrupa intenciones similares, genera candidatos de FAQs y permite que un usuario humano apruebe, rechace o edite cada propuesta antes de publicarla.

Planteamiento del reto y objetivos

Problema identificado

El problema principal consiste en que el volumen de conversaciones gestionadas por agentes de IA dificulta la identificación manual de patrones recurrentes. Sin un mecanismo automatizado, las oportunidades de mejora del agente quedan dispersas en los historiales de conversación y no se transforman de forma sistemática en FAQs reutilizables.

Objetivo general

Diseñar e implementar un prototipo funcional que detecte patrones conversacionales recurrentes en conversaciones históricas de Everwod y genere sugerencias de FAQs validadas por intervención humana.

Objetivos específicos

- Procesar conversaciones históricas almacenadas en PostgreSQL y normalizarlas para análisis semántico.
- Aplicar técnicas de limpieza, extracción de pares usuario-asistente, embeddings semánticos y clustering para agrupar consultas recurrentes.
- Generar preguntas y respuestas canónicas mediante un modelo local liviano, conservando respuestas prudentes y revisables.
- Permitir validación humana de las sugerencias mediante edición, aprobación, rechazo o marcación como revisión pendiente.
- Persistir métricas, corridas, ejemplos y decisiones para evitar duplicidad y mejorar ejecuciones futuras.
- Preparar el sistema para ejecución periódica mensual y despliegue como microservicio en Railway.

Metodología de desarrollo

La metodología aplicada fue incremental e iterativa. Primero se adaptó la base de datos restaurada desde el dump de Everwod; luego se construyó una capa de ingesta raw-to-clean; posteriormente se implementó el pipeline de NLP y generación; finalmente se integró un frontend de revisión humana y se modularizó el backend para dejar una arquitectura profesional.

Tabla 1

Metodología de implementación del prototipo.

Fase	Actividad	Resultado
1. Análisis del reto	Revisión de anexos, documentos, necesidades del aliado y lineamientos técnicos.	Alcance del microservicio: detección de patrones conversacionales y sugerencias de FAQs.
2. Preparación de datos	Restauración del dump en PostgreSQL y separación entre esquema crudo y esquema limpio.	Schemas everwod_raw y faq_mvp.
3. NLP y clustering	Limpieza, extracción de pares usuario-asistente, embeddings y agrupamiento.	Clusters de intenciones recurrentes por empresa.
4. Generación IA	Uso de Qwen local para formular pregunta/respuesta corta y prudente.	Candidatos FAQ con score, estado y evidencia.
5. Revisión humana	Frontend para aprobar, rechazar, editar y marcar needs_review.	Flujo human-in-the-loop.
6. Evaluación y documentación	Pruebas, métricas, smoke tests, lint y documentación técnica.	Prototipo verificable y listo para entrega.

Diseño arquitectónico del sistema propuesto

El sistema se diseñó como una arquitectura modular de microservicio, separando responsabilidades entre API, núcleo de configuración, ingesta, pipeline de IA, repositorio SQL y jobs programados. La API productiva unificada se expone desde app.api.main y concentra las rutas necesarias para el frontend y para operaciones administrativas.

La decisión de usar una API unificada simplifica el despliegue en Railway: en lugar de mantener tres servicios independientes para sugerencias, validación e ingesta, se despliega un único servicio faq-api y un job mensual independiente para ejecutar el pipeline de forma programada.

Figura 1

Arquitectura general propuesta para Everwod FAQ Intelligence.

Arquitectura propuesta: Everwod FAQ Intelligence

Vista simplificada del flujo de datos, IA y validación

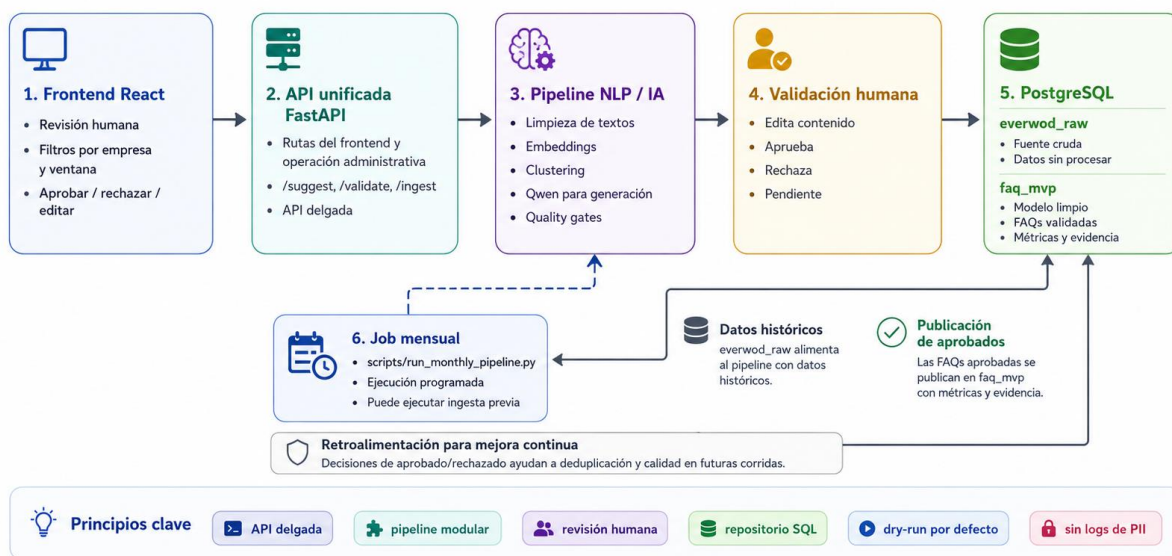


Figura 1. Arquitectura general propuesta para Everwod FAQ Intelligence

Módulos del backend

Tabla 2

Distribución modular del backend.

Módulo	Responsabilidad principal
app.api	Capa HTTP delgada. Expone endpoints /health, /workspaces, /suggest, /suggestions, /validations, /validate e /ingest.
app.core	Configuración por variables de entorno, conexión a base de datos, modelos Pydantic y utilidades comunes.
app.ingestion	Adaptador para leer conversaciones desde everwod_raw y escritor idempotente hacia faq_mvp.
app.pipeline	Limpieza, filtros, embeddings, clustering, generación Qwen, quality gates y orquestación.
app.repository	Persistencia SQL de corridas, métricas, clusters, candidatos, ejemplos, FAQs existentes y eventos de validación.
app.jobs	Ejecución periódica mensual con opción de ingesta previa.
frontend	Panel React/Vite para revisión humana de sugerencias.

Endpoints principales

Tabla 3

Contratos HTTP principales.

Endpoint	Uso en el prototipo
GET /health	Verifica estado del servicio y schemas activos.
GET /workspaces	Lista empresas disponibles para filtrar sugerencias.
POST /suggest	Ejecuta generación de nuevas sugerencias para una empresa y ventana temporal.

GET /suggestions	Consulta sugerencias generadas, por defecto de la última corrida exitosa.
GET /validations	Consulta historial de aprobaciones, rechazos y ediciones.
PATCH /suggestions/{candidate_id}	Permite editar pregunta/respuesta antes de aprobar.
POST /validate	Registra aprobación, rechazo o estado needs_review.
POST /ingest	Ejecuta ingesta segura desde everwod_raw, con dry_run por defecto.

Modelo de datos e ingesta de conversaciones históricas

La base de datos local se organizó en dos schemas: `everwod_raw` y `faq_mvp`. El schema `everwod_raw` representa la fuente cruda restaurada desde el dump entregado por la empresa, mientras que `faq_mvp` corresponde al modelo interno limpio del microservicio. Esta separación evita acoplar el pipeline de IA a la estructura operativa original y facilita adaptar la solución a una réplica o usuario readonly en producción.

Tabla 4

Separación entre fuente cruda y modelo limpio.

Schema	Propósito	Tablas relevantes
<code>everwod_raw</code>	Fuente cruda restaurada desde el dump de la empresa.	<code>workspaces</code> , <code>agents</code> , <code>agent_chats</code> , <code>chat_messages</code> , <code>agent_faqs</code> .
<code>faq_mvp</code>	Modelo interno normalizado para análisis y persistencia del pipeline.	<code>workspaces</code> , <code>agents</code> , <code>conversations</code> , <code>messages</code> , <code>existing_faqs</code> , <code>pipeline_runs</code> , <code>pipeline_metrics</code> , <code>question_clusters</code> , <code>faq_candidates</code> , <code>faq_candidate_examples</code> , <code>faq_validation_events</code> .

La ingesta es segura por defecto: el endpoint `POST /ingest` asume `dry_run=true` cuando el usuario no solicita explícitamente escritura. Con ello se pueden revisar métricas de lectura sin modificar la base. En las pruebas locales, el dry-run reportó 51 `workspaces`, 28 agentes, 7.967 chats, 35.124 mensajes disponibles en el dump, 212 FAQs existentes y extracción correcta de pares usuario-asistente en muestras controladas.

Pipeline de inteligencia artificial para generación de FAQs

El pipeline transforma conversaciones históricas en candidatos de FAQs mediante una secuencia de etapas: limpieza, extracción de evidencia, generación de embeddings, clustering, formulación de pregunta/respuesta canónica con LLM, validación de calidad y persistencia.

1. Limpieza y normalización

La limpieza elimina o reduce ruido conversacional: emojis, URLs, trazas internas, textos vacíos, información personal y expresiones que no aportan intención de FAQ. La limpieza no busca borrar el significado, sino preparar el texto para agrupar consultas similares y evitar que nombres propios o datos sensibles dominen el análisis.

2. Pares usuario-asistente

El sistema identifica mensajes de usuario y respuestas del asistente a partir del campo JSONB message. El formato soportado incluye role=user o role=assistant y content[].text.value. Con estos eventos se forman pares usuario→asistente que sirven como evidencia histórica para determinar si existe una consulta repetida y si hay una respuesta útil que pueda convertirse en FAQ.

3. Embeddings semánticos

Las preguntas candidatas se transforman en vectores semánticos mediante sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. Este modelo permite representar textos en español e inglés en un espacio vectorial, lo que favorece agrupar frases distintas con intención similar. Si el backend de embeddings no está disponible, el sistema conserva un fallback hash configurable para mantener el flujo de pruebas.

4. Clustering

El clustering agrupa consultas semánticamente cercanas. La configuración permite DBSCAN adaptativo, HDBSCAN opcional y fallback jerárquico. La distancia se calcula sobre embeddings, por lo que el agrupamiento no depende de que dos usuarios escriban exactamente la misma pregunta, sino de que expresen una intención parecida.

5. Generación con Qwen

Una vez detectado un cluster, el sistema usa Qwen/Qwen3-1.7B como modelo local para redactar una pregunta canónica corta y una respuesta prudente. El prompt instruye al modelo para no copiar saludos, nombres de clientes, emojis, trazas comerciales o respuestas personalizadas. Cuando la evidencia no es suficiente, el candidato se puede marcar como `needs_review` en lugar de publicarse directamente.

6. Quality gates

El módulo `quality.py` aplica estándares de calidad antes de persistir una sugerencia. Entre las validaciones se incluyen forma interrogativa, validez de respuesta canónica, alineación pregunta-respuesta, soporte por evidencia histórica, detección de intención mezclada, contaminación de salida del LLM y duplicidad contra FAQs previas.

Tabla 5

Principales controles de calidad aplicados a candidatos FAQ.

Criterio	Propósito
Pregunta canónica válida	Evita guardar frases que no son preguntas frecuentes o que no representan una intención clara.
Respuesta canónica válida	Evita respuestas demasiado largas, personalizadas, vagas o no publicables.

Alineación pregunta-respuesta	Comprueba que la respuesta responda a la pregunta formulada.
Soporte por evidencia	Determina si la respuesta está respaldada por conversaciones históricas.
Intención no mezclada	Evita clusters que mezclan temas incompatibles, por ejemplo, precios y horarios en una sola FAQ.
Repair pass	Permite corregir automáticamente pregunta/respuesta antes de rechazar o marcar como revisión.

Validación humana y frontend de revisión

El frontend se implementó como un panel de revisión humana. Permite seleccionar empresa, definir ventana temporal de análisis, generar nuevas sugerencias, buscar por texto, filtrar por estado, desplegar ejemplos del cluster y aprobar, rechazar o editar una FAQ antes de incorporarla a la base de conocimiento.

Figura 2

Vista del frontend de revisión con sugerencias, métricas e historial de validaciones.

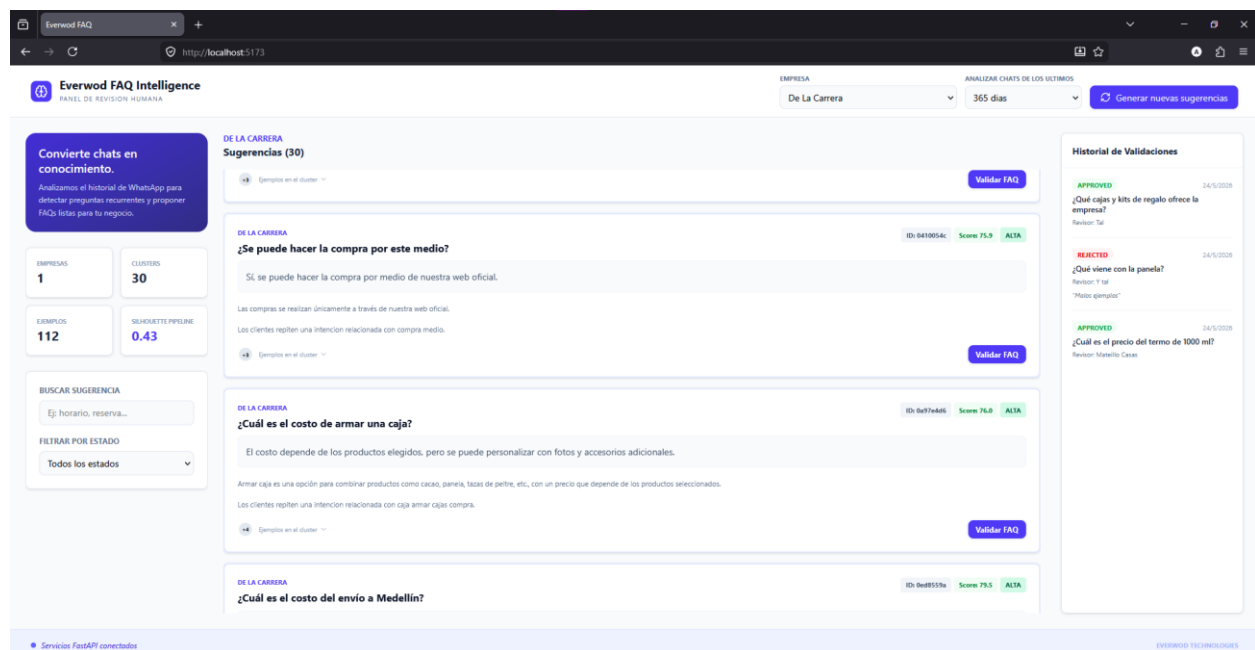
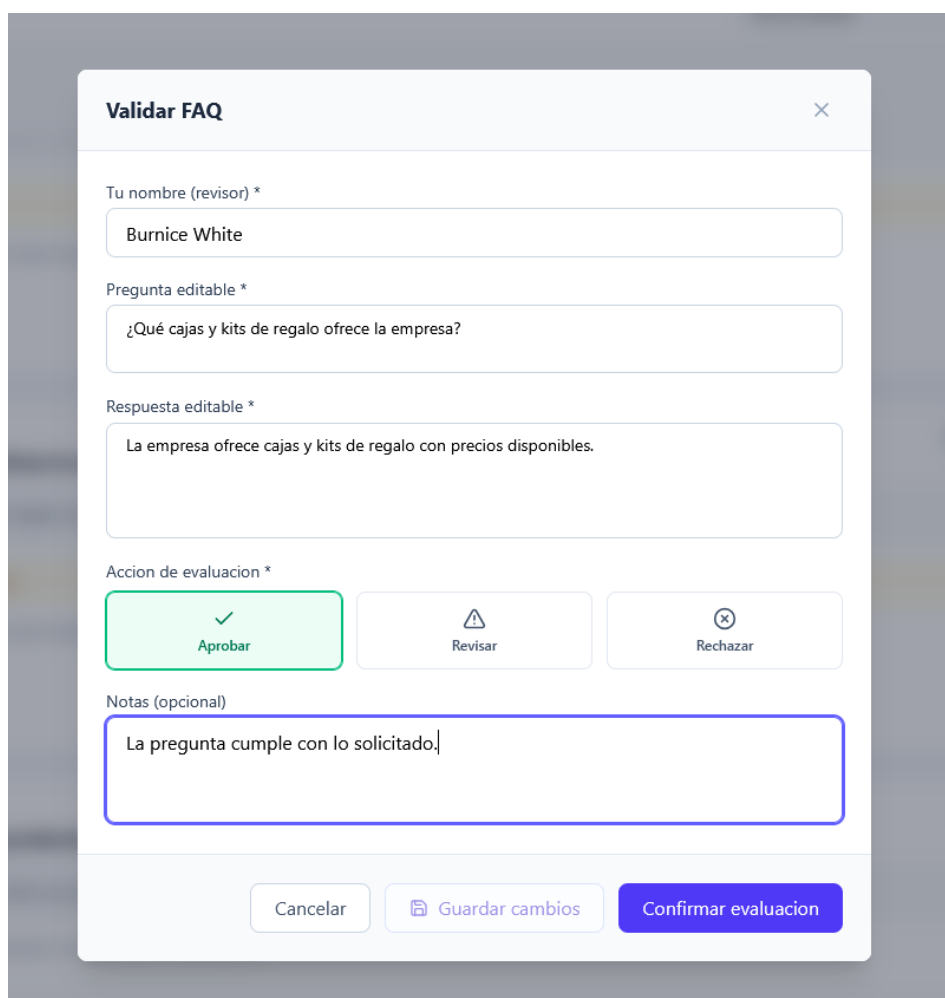


Figura 3

Usuario edita pregunta/respuesta y aprueba o rechaza la sugerencia.



The image shows a modal window titled "Validar FAQ" with a close button (X) in the top right corner. The form contains the following fields and controls:

- Tu nombre (revisor) ***: A text input field containing "Burnice White".
- Pregunta editable ***: A text input field containing "¿Qué cajas y kits de regalo ofrece la empresa?".
- Respuesta editable ***: A text input field containing "La empresa ofrece cajas y kits de regalo con precios disponibles."
- Accion de evaluacion ***: Three buttons: "Aprobar" (green with a checkmark icon), "Revisar" (light gray with a warning triangle icon), and "Rechazar" (light gray with an 'X' icon).
- Notas (opcional)**: A text input field containing "La pregunta cumple con lo solicitado|".

At the bottom of the modal are three buttons: "Cancelar", "Guardar cambios" (with a save icon), and "Confirmar evaluacion" (in blue).

La validación humana es un elemento clave del diseño: las sugerencias no se publican automáticamente sin revisión. Cuando un candidato se aprueba, se promueve a `existing_faqs`. Cuando se rechaza, queda registrado como candidato rechazado. En siguientes corridas, el sistema considera tanto FAQs aprobadas como rechazadas para evitar duplicados.

Implementación de ejecución periódica automatizada

El proyecto incluye ejecución programada mediante scripts y scheduler. La arquitectura recomendada para producción separa el servicio permanente `faq-api` del job mensual `faq-monthly-job`. El job puede ejecutarse con `python scripts/run_monthly_pipeline.py` y está preparado para correr de forma mensual, por ejemplo con una expresión cron `0 3 1 * *`.

La variable `FAQ_MONTHLY_RUN_INGEST` controla si antes de generar sugerencias se ejecuta la ingesta `raw`→`normalizado`. Por defecto permanece en `false` para evitar mutaciones no deseadas hasta que el equipo de Everwod confirme fuente oficial, permisos `readonly`, contrato JSONB y política de privacidad.

Evaluación experimental del desempeño

La evaluación se planteó desde tres dimensiones: calidad del agrupamiento, calidad de las FAQs generadas y utilidad operativa para el proceso de revisión humana. El sistema registra métricas por corrida en `pipeline_metrics` y las expone al frontend mediante tarjetas de empresas, clusters, ejemplos y silhouette pipeline.

Variables de configuración experimental

Tabla 6

Variables de configuración usadas para la evaluación experimental.

Variable	Valor usado	Interpretación
FAQ_CLUSTER_EPS	0.28	Radio de similitud para DBSCAN. Valores más bajos crean clusters más estrictos; valores más altos agrupan más consultas.
FAQ_MIN_CLUSTER_SIZE	3	Cantidad mínima de ejemplos para considerar un cluster como recurrente.
FAQ_MIN_CLUSTER_SUPPORT	0.50	Nivel mínimo de soporte histórico requerido para aceptar o revisar un cluster.
FAQ_MIN_CLUSTER_COHESION	0.45	Cohesión mínima entre ejemplos del cluster. Reduce mezclas de intenciones.
FAQ_MIN_GENERATION_CONFIDENCE	0.35	Confianza mínima de la generación LLM para persistir candidato.
FAQ_DUPLICATE_THRESHOLD	0.84	Umbral de similitud para evitar duplicados contra FAQs existentes o candidatos previos.
FAQ_CANDIDATE_HARVEST_MODE	Broad	Modo de cosecha de candidatos. Broad favorece recuperar más oportunidades para revisión humana.

FAQ_SAFE_FULL_ANALYSIS_LIMIT	50000	Límite seguro de mensajes para procesar completo si cabe; evita truncamientos innecesarios.
FAQ_ALIGNMENT_STRONG_SIMILARITY	0.56	Similitud para considerar fuerte la alineación entre pregunta canónica y cluster.
FAQ_ALIGNMENT_PARTIAL_SIMILARITY	0.42	Similitud para aceptar alineación parcial y enviar a revisión.
FAQ_SKIP_EXISTING / FAQ_SKIP_REJECTED	true / true	Evita regenerar FAQs ya aprobadas o rechazadas.

Silhouette pipeline

El silhouette es una métrica clásica de clustering que compara qué tan cerca está cada punto de su propio cluster frente a otros clusters. Su rango típico va de -1 a 1: valores altos indican clusters mejor separados; valores cercanos a 0 sugieren solapamiento; valores negativos pueden indicar asignaciones ambiguas. En el frontend se muestra como “Silhouette pipeline” porque corresponde al agrupamiento calculado por la corrida, no necesariamente al número final de sugerencias visibles después de los filtros de calidad.

Puede mostrarse una sola sugerencia final, pero conservar un silhouette válido si la corrida generó más clusters antes de aplicar filtros y deduplicación. Cuando no hay suficientes clusters comparables, el sistema muestra N/D.

Indicadores observados en pruebas locales

Tabla 7

Resultados funcionales y experimentales observados durante la validación local.

Prueba / corrida	Resultado observado	Lectura técnica
Dry-run de ingesta	51 workspaces, 28 agentes, 7.967 chats, 35.124 mensajes y 212 FAQs existentes detectadas.	La ingesta puede leer el dump raw sin escribir datos cuando dry_run=true.
Validación automatizada	72 pruebas backend aprobadas y lint frontend OK.	El sistema mantiene contratos de API, ingesta, sugerencias y validación.
Ejemplo De La Carrera	30 sugerencias visibles, 112 ejemplos y silhouette pipeline aproximado de 0.43.	El pipeline pudo generar múltiples oportunidades de FAQ para una empresa concreta.
Ejemplo ADN Box	1 sugerencia, 3 ejemplos y silhouette pipeline aproximado de 0.69.	Aunque haya una sugerencia final, la métrica puede venir de la corrida de clustering.
Historial de validaciones	Aprobaciones y rechazos visibles en el panel.	El sistema registra decisiones humanas y las usa para evitar duplicados futuros.
Ejecución global por empresas	Se ejecutó run_all_workspaces.ps1 para correr el pipeline por empresa. En el dump final quedaron registrados 170 candidatos FAQ, 637 ejemplos asociados y 223 clusters. De los candidatos, 129 quedaron como needs_review, 40 como pending y 1 como rejected.	El pipeline pudo ejecutarse de forma masiva por workspace y persistir resultados en PostgreSQL. La mayoría de los candidatos quedó pendiente de revisión, lo que confirma que el sistema no publica automáticamente, sino que prioriza validación humana, evidencia y control de calidad.

Precisión, reducción de ambigüedad y mejora en tasa de resolución

La precisión del prototipo se evaluó mediante criterios de aceptación: candidatos high confidence, candidates needs_review, rechazo por baja evidencia, rechazo por mala alineación y duplicados omitidos. En lugar de publicar automáticamente todos los clusters, el sistema separa candidatos publicables de candidatos que requieren revisión, reduciendo el riesgo de incorporar FAQs incorrectas.

La reducción de ambigüedad se logra al transformar múltiples mensajes de clientes en una pregunta canónica breve y una respuesta estándar. Por ejemplo, consultas dispersas sobre domicilio, precio, disponibilidad o agendamiento se convierten en una FAQ estructurada que puede ser revisada y aprobada. La mejora potencial en tasa de resolución proviene de incorporar FAQs aprobadas a existing_faqs, lo que permite que el agente consulte conocimiento validado en ejecuciones futuras.

Calidad, seguridad y mantenibilidad

La calidad del software se alineó con criterios de ISO/IEC 25010: mantenibilidad, fiabilidad, eficiencia, compatibilidad, usabilidad, seguridad y testabilidad. La modularización real separa API, pipeline, repositorio, ingesta y configuración, facilitando pruebas y mantenimiento.

Tabla 8

Alineación con ISO/IEC 25010.

Característica ISO/IEC 25010	Aplicación en el proyecto
Mantenibilidad	Paquete app/ dividido por responsabilidad y API delgada.
Fiabilidad	Tests automatizados, dry-run de ingesta y persistencia idempotente.
Eficiencia	Modelo local liviano, Docker CPU-only y job mensual separado de la API permanente.
Compatibilidad	Endpoints legacy conservados y API unificada para Railway.
Usabilidad	Panel visual para revisar sugerencias, métricas y ejemplos.
Seguridad	No logging de PII, separación raw/clean.
Funcionalidad	72 pruebas backend, lint frontend y pruebas unitarias de ingesta con mocks.

Resultados obtenidos

- Se implementó un microservicio funcional capaz de analizar conversaciones históricas por empresa y ventana temporal.
- Se creó una capa de ingesta segura desde everwod_raw hacia faq_mvp, con dry-run por defecto.
- Se integraron embeddings semánticos multilingües, clustering y generación con Qwen local.
- Se desarrolló un frontend de revisión humana con filtros, métricas, validación y edición.
- Se implementó deduplicación contra FAQs existentes, candidatos previos, aprobados y rechazados.
- Se agregó ejecución mensual programable y documentación para despliegue en Railway.
- Se validó el sistema mediante pruebas automatizadas, smoke tests y pruebas manuales de frontend.

Conclusiones

El proyecto cumple con el objetivo de transformar conversaciones históricas en conocimiento accionable para mejorar agentes de IA. La solución no se limita a generar texto con un LLM: incorpora limpieza, evidencia histórica, embeddings, clustering, quality gates, validación humana, persistencia y control de duplicados.

La arquitectura modular facilita el despliegue y mantenimiento del sistema. La separación entre `everwod_raw` y `faq_mvp` permite adaptar la solución a la infraestructura real de Everwod sin acoplar el pipeline al dump original. Asimismo, la API unificada reduce complejidad operativa al concentrar sugerencias, validación e ingesta en un solo servicio.

La evaluación experimental muestra que el sistema puede generar sugerencias útiles por empresa, registrar métricas de calidad y permitir que el usuario humano decida qué FAQs deben ser publicadas. El enfoque human-in-the-loop es adecuado para un contexto empresarial donde las respuestas deben ser prudentes, verificables y alineadas con políticas del negocio.

Recomendaciones y trabajo futuro

- Confirmar con Everwod la fuente oficial de conversaciones, permisos readonly o réplica y contrato JSONB de producción.
- Definir una política formal de retención, anonimización y acceso a datos sensibles.
- Agregar un selector de corridas históricas en el frontend para comparar resultados por run_id.
- Incorporar evaluación humana cuantitativa con tasa de aceptación, tasa de rechazo y precisión percibida por empresa.
- Medir impacto real sobre resolución del agente después de cargar FAQs aprobadas en producción.
- Explorar modelos locales alternativos si se requiere mayor calidad de redacción con bajo costo computacional.